A TECHNICAL DOCUMENTARY

The Shabo *Documentary*

The Fast-Fast Interview Coach

A technical journal tracing how one CS student in Tarkwa, Ghana, built an AI interview coach (Simple FullStack with AI/ML) — through Kaggle data dumps, async traffic jams, CORS errors, mobile layout battles, and a 2 AM confrontation with Error 429.

Author Fauzan Abdul-Razak

Institution UMaT, Tarkwa, Ghana — 3rd Year CSE

Context AI Course Group Project

Stack Kaggle / FastAPI / React + Vite / Google Gemini / Render / Vercel

C O N T E N T S

INTRODUCTION

The "Tomorrow" Problem 3

The Architect Behind the Screen 3

CHAPTER ONE

Mining the Conversation 5

CHAPTER TWO

The Heartbeat and the Brain 7

CHAPTER THREE

The Face, The Bouncer, and The Traffic Jam 8

CHAPTER FOUR

The Great Mobile Squish 10

CHAPTER FIVE

The Night of the 429 Ghost 12

EPILOGUE

The Save State 14

APPENDIX

The Journey Map 15

INTRODUCTION

The "Tomorrow" Problem

There is a universal panic in the professional world. You apply for a job, you wait for weeks, and then you get the email: *"We'd like to interview you... tomorrow."*

Most people don't have time to prepare for interviews. You can't read a 300-page book on interview psychology the night before. You need a quick rehearsal. **You need something fast.**

This project started as an assignment for our 3rd-year AI course at UMaT. We were put into groups, but I chose to be the one to actually build the thing from the ground up. I knew exactly what I wanted it to be.

I called it **Shabo.**

> The name comes from the Ghanaian slang term "shabo-shabo," which means "fast-fast" or "quick." That wasn't just a catchy name; it became the entire engineering philosophy for the project. The app had to be fast to load. The AI had to be fast to reply. The user had to be able to jump in, rehearse, get graded, and get out because they have little to no time for preparations. Shabo-shabo.

This documentary is the record of how that happened. Every bug, every server crash, and every line of code it took to make a machine understand a human interview.

The Architect Behind the Screen

Hi, I'm **Fauzan,** a third-year Computer Science and Engineering student at UMaT in Tarkwa. If you know Tarkwa, you know it's a mining town. The whole city is built on the philosophy of extraction — digging through the dirt, dealing with the pressure, and pulling out the gold. Down in the CS department, we do the exact same thing, just with data.

Being in your third year is a strange kind of pressure. The academic bubble starts to pop, and the reality of the actual job market is suddenly staring you in the face. You look around the lecture halls and hostels, and you see brilliant coders. These are people who can write complex algorithms in their sleep, but who will completely freeze up when a recruiter asks a simple question like, *"Tell me about a time you failed."*

When this project was assigned in our AI class, our group had a choice. We could build something purely theoretical, write a boring academic paper, get our grade, and go to sleep.

But I wanted the keyboard. I volunteered to build it.

I didn't want to just study Artificial Intelligence on a whiteboard; I wanted to wrestle with it. I wanted to build something that the guy sitting in the room next to me could actually use to prep for a life-changing phone call.

I am not a tenured professor or a tech-giant CEO. I am a student in Ghana with a laptop, a spotty internet connection, and a deadline. And as it turns out, that was the exact kind of pressure required to build something *shabo-shabo*.

CHAPTER ONE

Mining the Conversation

You can't just tell a computer, "Act like an interviewer." A machine is empty; it only knows what you feed it. To build Shabo, I needed to feed it the DNA of real human conversations.

I started on **Kaggle,** the massive online database where data scientists share raw information. I went hunting and finally downloaded a heavy dataset of raw interview transcripts.

But having the data is only ten percent of the work. The transcripts were a disaster.

> **WHAT WENT WRONG**
> I launched my Jupyter environment and essentially became a digital janitor. The data was full of junk — random timestamps, interviewer tags, weird formatting, and a thousand "umms" and "ahhs." If I fed that to the AI, it would learn bad habits.

I opened my physical notebook, grabbed a pen, and mapped out the logic. Then, I turned to **Regex.**

> **PLAIN ENGLISH: WHAT IS REGEX?**
> Imagine opening a 1,000-page document and trying to manually delete every time someone cleared their throat, said "umm," or a weird computer glitch typed a random #. You'd lose your mind.
>
> **Regex (Regular Expressions)** is basically a magical, super-powered "Find and Replace." You write a cryptic little code like `[^a-zA-Z\s]`, and it acts like a digital precision tool, hunting down and wiping out exactly the junk you want gone in a fraction of a second.

Once the text was clean, I hit the real Computer Science obstacle: Computers don't speak English. They speak Math.

I had to use a statistical formula called **TF-IDF.**

> **PLAIN ENGLISH: WHAT IS TF-IDF?**
> Think of it like this: if I hand a computer a sentence, it just sees squiggly shapes. TF-IDF (Term Frequency-Inverse Document Frequency) is a math trick that weighs words.
>
> Imagine you're reading a book about UMaT. The word "the" appears a million times, but it's useless. The word "engineering" appears less, but it tells you the whole story. TF-IDF is like the computer taking a highlighter and marking only the "heavy" words. It takes messy human conversation and turns it into a neat mathematical matrix.

> **NOTEBOOK ENTRY**
> Trained and fed the model. The Jupyter kernel is stable. I sat back and looked at the screen. It wasn't an app yet. It had no buttons and no interface. But the foundation was laid. I could breathe — I didn't want to deal with synthetic data.

CHAPTER TWO

The Heartbeat and the Brain

A Jupyter notebook is great for science experiments, but it's not a product you can share with your friends. I needed to build an engine that could run on its own.

I chose to build the backend with **FastAPI.**

> PLAIN ENGLISH: WHAT IS FASTAPI?
> Think of it as the kitchen of a really efficient restaurant. It takes the orders (when a user asks a question) and prepares the food (the AI's response) incredibly fast.
>
> But a kitchen needs someone to handle the customers. So, I brought in Uvicorn to act as the server — the bouncer at the front door, managing the traffic so the kitchen doesn't catch fire when 100 people show up at once.

> NOTEBOOK ENTRY
> I remember running the startup command for the first time in the terminal. The laptop fan whirred, the text scrolled green, and the server output: Application startup complete. Shabo had a heartbeat.

But a beating heart isn't enough; it needed a brain. My math model was good at recognising patterns, but a real interview coach needs to *reason*. It needs to listen to your answer and say, "Wait, you missed the 'Action' part of the STAR method." To get that level of intelligence, I installed the **Google AI SDK.**

This was the ultimate brain transplant. It was the bridge between my little local setup in Tarkwa and one of the most powerful AI infrastructures on the planet. I generated my API keys — which is basically a VIP access card that lets my code talk directly to Google's AI — and plugged them into my FastAPI kitchen.

Suddenly, the code wasn't just filtering words anymore. **It was thinking.** It was ready to talk. But before it could, I had to build it a face.

CHAPTER THREE

The Face, The Bouncer, and The Traffic Jam

Having a super-smart AI running in a black terminal window is cool for about five minutes. After that, you realise it's basically just a brain floating in a jar. Shabo needed a face.

I spun up the frontend using **Vite** and **React.**

> **PLAIN ENGLISH: WHAT IS VITE?**
> Think of Vite (which means "fast" in French) like a hyperactive assistant. Usually, when you write code for a website, you have to wait for the computer to bundle all the files together before you can see what it looks like. Vite skips the wait. You type a line of code, hit save, and the website updates instantly. The perfect tool for the shabo-shabo philosophy.

I wanted the interface to look premium. Not like a boring academic project, but like something a tech lead would use. I designed a dark, glowing "Glassmorphism" look, and wrapped the interview screen inside a sleek MacBook mockup.

It looked beautiful. It was time to connect the React face to the FastAPI brain. I hit "Run" and...

> **ERROR**
> Blocked. Red text flooded my console. I had just met the first boss of web development: CORS (Cross-Origin Resource Sharing).

> **PLAIN ENGLISH: WHAT IS CORS?**
> Imagine your frontend and your backend are trying to hang out at a club. The frontend walks up and says, "Hey, I need some interview questions!" But CORS is a giant, overly aggressive security bouncer at the door. He looks at the frontend and says, "I don't know you. You aren't from around here. Access denied."
>
> To fix it, I had to go back into my Python code and write a VIP pass — a middleware script — telling the bouncer, "Chill out, man. The localhost frontend is with me. Let him in."

> **VICTORY**
> They finally shook hands. I typed in a test prompt, and Shabo replied. It worked!

So, I asked it a second question. ...Nothing. The loading spinner just kept spinning. The server had completely frozen.

I was officially stuck in **The Async Traffic Jam.** This was one of those bugs that makes you want to close your laptop and rethink your major. Everything was working perfectly for *one* question, but the second I tried to generate another set of questions, Render would drop the connection. The server was timing out.

I dug through the code and realised I had walked right into a classic FastAPI trap. I had defined my Python functions using `async def`.

> **PLAIN ENGLISH: THE ASYNC TRAFFIC JAM**
> Imagine FastAPI is a chef in a kitchen. Async means the chef is a master multitasker. If he puts a pizza in the oven, he doesn't just stand there staring at it; he goes and chops onions for the next order.
>
> But the Google Gemini AI call was a synchronous task. That means it demands full attention. By telling my async chef to handle a synchronous task, I basically forced him to put the pizza in the microwave, lock the kitchen door, and just stand there staring at the spinning plate while the whole restaurant outside burned down waiting for their food. The second request couldn't get in because the chef was frozen.
>
> **The Fix:** I had to swallow my pride, delete the `async` keyword, and change it back to a normal `def`. In FastAPI, doing this automatically tells the system: "Hey, this task takes a while. Hand it off to a background worker so the main chef can keep taking orders."

> **NOTEBOOK ENTRY**
> I saved the file. Restarted the server. Request 1: Shabo-shabo. Request 2: Shabo-shabo. Request 3: Still flying. The traffic jam was cleared. The brain and the face were finally working in perfect harmony. Now, it was time to put it on the real internet.

CHAPTER FOUR

The Great Mobile Squish

So, Shabo was officially live on the internet. My Vercel link was working, and it was talking to my Render backend perfectly.

You know that feeling when you try on a brand-new outfit in your bedroom mirror, and you look like a million bucks, but then you catch a glimpse of yourself in a store window and you look terrible?

That was me looking at Shabo on my phone.

> **WHAT I SAW**
> On my wide laptop screen, the UI was a masterpiece. But the moment I opened the live link on my phone, it was a tragedy. The sleek MacBook mockup I spent hours designing looked like it had been run over by a trotro. The text was overlapping, the buttons were shrinking into oblivion, and the cool "Logic Score" charts were violently squishing each other to fit on the screen.

I was officially drafted into the CSS War.

> **PLAIN ENGLISH: WHAT IS CSS?**
> Think of it as the clothing for your code. HTML is the skeleton, and CSS tells the skeleton how to dress itself.
>
> I spent the next three hours in the trenches, tweaking numbers by 1%. I had to write "Media Queries" — which is basically a rule that tells the website, "Hey, if you notice the screen is super small, panic and stack everything vertically like pancakes!"

And then, I met my arch-nemesis: **The iOS Auto-Zoom Bug.** Every time I tapped the text box on my phone to type an interview answer, the whole screen would abruptly zoom in, ruining the layout.

It turns out, iPhones have a hidden, stubborn rule: if your font size is smaller than `16px`, the phone assumes the text is too small to read and violently zooms in to "help" you. I went into the code, forced the font to exactly `16px`, and pushed the update to Vercel.

> **VICTORY**
> The zooming stopped. The buttons were chunky and perfect for thumbs. I leaned back in my chair, staring at the ceiling. I had just spent two hours arguing with a phone about font sizes.

CHAPTER FIVE

The Night of the 429 Ghost

It was late. I was tired, but the good kind of tired. The site was live. It was mobile-responsive. I was running through test after test. Generate questions. Answer. Get graded. Generate again. I was feeling like a genius. I was ready to drop the link in our group chat and go to sleep.

I clicked "Generate" one more time.

> **ERROR**
> `500 Internal Server Error`
> My stomach dropped. I rushed to my Render server logs. Right there in the console, glowing in red, was the killer:
> `Error 429: RESOURCE_EXHAUSTED`

What did that mean? Well, to make Shabo smart, I was using an experimental "Lite" AI model on Google's free tier. The AI was basically saying, *"Listen buddy, you've asked me 20 questions today. That's your limit. I'm going home. Come back tomorrow."*

I was locked out of my own creation. I had built a beautiful sports car, driven it around the block twice, and completely ran out of gas.

> **NOTEBOOK ENTRY**
> It was 2:00 AM. The presentation for our AI course was creeping closer. I could have just closed the laptop and gone to sleep. But you don't call a project "Shabo" and then tell people they have to wait 24 hours for it to work.

I frantically researched the AI documentation. I found the escape hatch: **Gemini 3 Flash.** It was a much more stable model — and more importantly, the free tier allowed *thousands of requests a day,* not just 20.

I opened my backend code, deleted the old model name, typed in `gemini-3-flash,` and pushed the update to Render. My heart was pounding while I waited for the server to rebuild.

I picked up my phone and reloaded the app. I clicked "Start."

> **VICTORY**
> Boom. Questions loaded instantly. I clicked it again. Boom. Graded. Fifty requests later, it was still flying. I had gone from a 20-request limit to an absolute powerhouse. Shabo wasn't just fixed; it was bulletproof.

EPILOGUE

The Save State

When I look back at the lines of code, the notebook scribbles, and the empty coffee cups, I don't just see an AI course assignment for a grade.

I see the exact moment I realised that building software isn't about following a perfect blueprint. **It's about surviving the chaos.**

It's about being a digital janitor on Kaggle, wrestling with an async kernel, fighting a club-bouncer named CORS, arguing with Apple's font rules, and refusing to give up when the server tells you you're out of resources.

> NOTEBOOK ENTRY
> The guys in my group got to see the polished, working product. But this documentary is for the messy middle.

So if you are reading this right now — whether you're a fellow CS student drowning in the pressure of your 3rd year in Tarkwa, or just me, reading this years from now to remember what it felt like — remember this:

The pressure is real. The bugs will make you want to throw your laptop out the window. But when you finally hit that deploy button, and the screen comes alive, and you realise you just taught a machine how to think... man, there is nothing else like it.

APPENDIX

The Journey Map

A summary of the full development path — from raw data to a deployed, bulletproof product.

Step	What Happened	Phase
01	**Kaggle Data Hunt** Downloaded raw interview transcripts. Became a digital janitor — timestamps, stray tags, and a thousand "umms" had to go.	DATA COLLECTION
02	**Regex + TF-IDF** Used Regular Expressions to scrub dirty data. TF-IDF converted human words into a mathematical matrix the AI could process.	NLP
03	**FastAPI Backend** Built the kitchen. FastAPI as chef, Uvicorn as the front-door manager. Plugged in the Google AI SDK — the brain transplant.	BACKEND
04	**Vite + React Frontend** Gave Shabo a face. Dark Glassmorphism UI with a MacBook mockup. Instant hot-reload via Vite.	FRONTEND
05	**CORS Battle + Async Fix** Wrote a middleware VIP pass to satisfy CORS. Removed the async keyword to unfreeze the kitchen chef.	DEBUGGING
06	**Mobile CSS War** Wrote Media Queries to stack the layout on small screens. Defeated the iOS Auto-Zoom Bug with a 16px font-size rule.	RESPONSIVE DESIGN
07	**429 Error Ghost** 2 AM. Error 429: RESOURCE_EXHAUSTED. The free-tier lite model quit at 20 requests. Researched and found the escape hatch.	CRISIS
08	**Gemini 3 Flash — Live** Swapped in Gemini 3 Flash. Thousands of requests per day. Deployed on Vercel + Render. Shabo was live and bulletproof.	DEPLOYED

"When you finally hit that deploy button, and the screen comes alive, and you realise you just taught a machine how to think... man, there is nothing else like it."

— Fauzan
UMaT, Tarkwa, Ghana / 3rd Year Computer Science & Engineering

SHABO DOCUMENTARY